

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 – Redes de Computadoras

Sección 10

Ing. Kevin Velásquez



Proyecto No. 1

Uso de un protocolo existente

Angie Nadissa Vela López, 23764

Guatemala, 20 de agosto de 2026

1. Introducción

En este proyecto se desarrolló un chatbot de consola que funciona como anfitrión de varios servidores MCP. El chatbot se comunica con un modelo de lenguaje mediante la API de Anthropic y, a través de clientes MCP implementados manualmente, puede utilizar servidores para interactuar con el sistema de archivos, Git y una aplicación propia orientada al área de farmacias.

Uno de los principales objetivos del proyecto fue comprender el funcionamiento del protocolo más allá de utilizar una implementación existente. Por esta razón, la comunicación JSON-RPC y el ciclo de vida de MCP fueron implementados manualmente, sin utilizar SDKs que abstraieran el funcionamiento del protocolo.

El servidor desarrollado para este proyecto corresponde a un caso de uso de una cadena de farmacias. Este servidor permite evaluar síntomas, buscar medicamentos, revisar interacciones y alergias, consultar el inventario de diferentes sucursales y crear órdenes de compra, incluyendo reglas para evitar la compra de medicamentos de prescripción cuando no se proporciona una receta.

El Model Context Protocol (MCP) es un protocolo abierto propuesto por Anthropic en noviembre de 2024 que estandariza la forma en que los modelos de lenguaje (LLMs) acceden a herramientas externas. Antes de MCP, cada proveedor de LLM definía su propio mecanismo de integración de herramientas, lo que obligaba a reimplementar cada herramienta para cada modelo. MCP resuelve este problema separando claramente tres roles: el servidor (expone herramientas), el cliente (mantiene la conexión con un servidor) y el anfitrión (la aplicación de IA que coordina varios clientes).

Este documento describe el estado del proyecto correspondiente a la primera fase de la entrega: el chatbot general (conexión con el LLM, contexto de conversación y bitácora de interacciones MCP), el uso de los servidores MCP oficiales de Filesystem y Git, y el servidor MCP propio para el caso de uso de farmacia, ejecutado de forma local.

2. Objetivos

2.1 Objetivo General

Implementar un chatbot capaz de utilizar múltiples herramientas mediante el Model Context Protocol (MCP), desarrollando manualmente la comunicación basada en JSON-RPC y conectando el sistema con un Large Language Model mediante su API.

2.2 Objetivos específicos

- Implementar un protocolo en base a los estándares.
- Comprender el propósito del protocolo MCP.
- Comprender cómo funcionan los servicios del protocolo MCP.
- Implementar servidores MCP locales y remotos
- Comprender como interactuar con un LLM a nivel de la API.

3. Marco teórico

3.1 Large Language Models

Los Large Language Models son modelos de inteligencia artificial entrenados con grandes cantidades de información para procesar y generar lenguaje natural. Una de sus principales características es su capacidad para interpretar las instrucciones proporcionadas por un usuario y generar respuestas coherentes de acuerdo con el contexto disponible.

Sin embargo, un LLM no tiene acceso directo a todos los sistemas externos que una aplicación podría necesitar. Por ejemplo, el modelo no puede consultar directamente el inventario de una farmacia, modificar un archivo local o ejecutar un comando de Git únicamente por conocer la respuesta correcta. Para realizar este tipo de acciones necesita utilizar herramientas.

En este proyecto, el LLM funciona como el componente encargado de interpretar las solicitudes del usuario y decidir cuándo necesita utilizar una herramienta. Las acciones reales son ejecutadas por los servidores MCP.

3.2 Model Context Protocol

Model Context Protocol (MCP) es un protocolo diseñado para facilitar la comunicación entre aplicaciones que utilizan modelos de lenguaje y herramientas externas.

El protocolo define una arquitectura en la que participan principalmente tres componentes: el anfitrión, el cliente y el servidor.

El anfitrión es la aplicación que utiliza el modelo de lenguaje y coordina las diferentes herramientas disponibles. En este proyecto, el chatbot de consola funciona como anfitrión.

El cliente MCP es el componente encargado de mantener la comunicación entre el anfitrión y un servidor específico. Un mismo anfitrión puede tener varios clientes, uno por cada servidor al que se conecta.

Finalmente, el servidor MCP proporciona las herramientas que pueden ser utilizadas por el modelo. En este proyecto se utilizaron servidores existentes para Filesystem y Git, además de un servidor propio para el caso de uso de farmacia.

Esta separación permite que el servidor de una herramienta no dependa directamente del modelo de lenguaje utilizado por el anfitrión.

3.3 JSON-RPC

MCP utiliza JSON-RPC para estructurar la comunicación entre clientes y servidores. JSON-RPC permite representar diferentes tipos de mensajes utilizando objetos JSON.

En el proyecto se implementaron manualmente los principales tipos de mensajes necesarios para la comunicación:

- Solicitudes.
- Respuestas.
- Notificaciones.
- Respuestas de error.

Cada solicitud posee un identificador que permite relacionar posteriormente la respuesta con la operación que la originó.

Para la comunicación mediante stdio, cada mensaje JSON-RPC se transmite como una línea independiente. De esta forma, el cliente puede enviar una solicitud y posteriormente esperar la respuesta correspondiente.

Una de las decisiones importantes del proyecto fue implementar esta comunicación directamente, en lugar de utilizar una biblioteca que ocultara el funcionamiento de JSON-RPC o de MCP. Esto permitió comprender de manera más directa cómo se estructura y transporta la información.

4. Caso de uso implementado

El caso de uso de industria elegido es un asistente conversacional para una cadena de farmacias. A partir de los síntomas que describe el usuario, el asistente evalúa la gravedad del cuadro, identifica señales de alarma que requieren atención médica inmediata, recomienda medicamentos de venta libre (OTC) apropiados, verifica alergias, consulta existencias por sucursal y permite generar un pedido. Los medicamentos que requieren receta se rechazan a menos que se proporcione un identificador de receta válido.

Esta es una funcionalidad sensible por naturaleza (el asistente literalmente recomienda un tratamiento) por lo que las reglas de seguridad (señales de alarma, restricción de medicamentos con receta, disclaimer médico) se implementaron dentro del propio servidor MCP y no únicamente como instrucciones en el prompt del modelo, de forma que se cumplen sin depender de que el LLM las respete.

5. Arquitectura del sistema

5.1 Arquitectura por capas

El proyecto está organizado en capas con dependencias que solo apuntan hacia abajo, es decir, cada capa únicamente conoce a la que tiene inmediatamente debajo. Esto permite probar y razonar sobre cada capa de forma aislada, y hace evidente en el propio código dónde vive cada responsabilidad del protocolo.

Capa	Responsabilidad	No conoce
src/protocol/	JSON-RPC 2.0 puro: mensajes, errores, framing	MCP ni el LLM
src/mcp/	Ciclo de vida MCP y transportes (stdio)	El LLM
src/llm/	Cliente crudo de la API de mensajes de Anthropic	MCP
src/host/	Registro de tools, conversación, agent loop, orquestación	La terminal
src/observability/	Bitácora y renderizado de frames MCP	—
src/servers/pharmacy/	Servidor MCP propio: dispatcher y lógica de dominio	El transporte usado
src/main.py	REPL de terminal	JSON-RPC directamente

5.2 Stack tecnológico

- Python 3.11 con asyncio para toda la concurrencia (subprocesos, E/S de red).
- httpx para las llamadas HTTP crudas a la API de Anthropic, sin el SDK oficial de Anthropic.
- Sin ninguna dependencia de MCP/FastMCP: JSON-RPC y el ciclo de vida MCP están implementados desde cero en src/protocol/ y src/mcp/.

- pytest y pytest-asyncio para la suite de pruebas automatizadas.
- Los servidores MCP oficiales (Filesystem, Git) se ejecutan como procesos hijos vía npx y pip respectivamente; su implementación interna no es parte de este proyecto.

5.3 Estructura de directorios

```
src/  
  protocol/      JSON-RPC 2.0 escrito a mano (mensajes, errores, framing)  
  mcp/          Ciclo de vida MCP sobre JSON-RPC: cliente, modelos, transportes  
  llm/          Cliente crudo de la API de mensajes de Anthropic  
  observability/ Bitácora de frames MCP (JSONL) y renderizado en terminal  
  host/        El anfitrión: configuración, registro de tools, conversación, agent loop  
  servers/pharmacy/ El servidor MCP propio (lógica de dominio, datos, dispatcher)  
  main.py       Punto de entrada: REPL de terminal  
  tests/        Pruebas unitarias e integración  
  docs/         Especificación del servidor, escenarios de demo, este informe  
  config/       Registro declarativo de servidores MCP (servers.json)
```

6. Funcionalidades implementadas

Esta sección demuestra los requisitos del enunciado correspondientes al funcionamiento general del chatbot y a la primera parte de servidores MCP.

6.1 Conexión con un LLM a nivel de su API

Implementado en `src/llm/anthropic_api.py`. La clase `AnthropicClient` construye a mano el cuerpo JSON de la solicitud (`model`, `max_tokens`, `system`, `messages`, `tools`) y lo envía con `httpx.AsyncClient` directamente a `https://api.anthropic.com/v1/messages`, usando los encabezados `x-api-key` y `anthropic-version`. No se usa el SDK `anthropic`, por lo que la forma

exacta de la solicitud y la respuesta permanece visible en el código. Se verificó con la pregunta (“¿Quién fue Alan Turing?”) usando una clave de API real, obteniendo una respuesta correcta con stop_reason “end_turn”.

6.2 Mantener contexto en una sesión

Implementado en src/host/conversation.py. La clase Conversation acumula el historial completo de mensajes en el formato que espera la API de Anthropic (role + bloques de contenido) y lo recorta a un máximo configurable de mensajes antes de cada envío, para no hacer crecer la ventana de contexto sin límite. Se verificó igualmente con la pregunta “¿Quién fue Alan Turing?” seguida de “¿En qué año nació?”, fue respondida correctamente sin repetir el nombre, demostrando que el historial se mantiene entre turnos.

6.3 Bitácora de todas las interacciones con servidores MCP

Implementado en src/observability/mcp_log.py y src/observability/render.py. Cada MCPClient recibe un callback on_frame que se invoca por cada mensaje JSON-RPC enviado o recibido (solicitudes, respuestas y notificaciones, de los tres servidores conectados), antes de cualquier otro procesamiento. Cada frame se agrega a un buffer en memoria (los últimos 500) y se escribe como una línea JSON en logs/mcp-<timestamp>.jsonl. El comando /log del REPL muestra los últimos n frames capturados, con dirección, servidor, tipo de mensaje (solicitud/respuesta/notificación/error) y el cuerpo JSON completo. Se verificó ejecutando el REPL real, encadenando /servers, /tools y /log tras una conversación con tool calling.

6.4 Servidores MCP locales oficiales (Filesystem y Git)

El host se conectó exitosamente a los dos servidores MCP oficiales de Anthropic vía transporte stdio (config/servers.json), el Filesystem MCP server (modelcontextprotocol/server-filesystem, lanzado con npx) y el Git MCP server (mcp-server-git, instalado con pip). Ambos exponen su catálogo completo de herramientas al iniciar (14 y 12 respectivamente), confirmando que el ciclo de vida MCP implementado a localmente funciona con implementaciones reales y el servidor propio.

Respecto al escenario, en las pruebas se le pidió al asistente crear un archivo README.md dentro de un repositorio, agregarlo al índice y confirmarlo (realizar un commit). El asistente encadenó correctamente todos los procesos, se verificó directamente con git log en el repositorio de prueba. Una limitación que encontré es que la versión publicada de mcp-server-git no expone una herramienta git_init, por lo que el repositorio debe existir de antemano. Por ende, el chatbot no puede crear el repositorio por sí mismo usando únicamente estos dos servidores oficiales. El escenario de demostración se ajustó en consecuencia y queda documentado en docs/demo-scenarios.md.

6.5 Servidor MCP propio con caso de uso de industria

El servidor Pharmacy MCP (src/servers/pharmacy/) está completamente implementado, probado y documentado para ejecución local sobre stdio. Su especificación completa se detalla a continuación y en docs/pharmacy-mcp-spec.md.

7. Especificación del servidor Pharmacy MCP

El servidor se ejecuta con `python -m src.servers.pharmacy.stdio_server`. Su dispatcher (`src/servers/pharmacy/core/dispatcher.py`) es independiente del transporte, recibe un mensaje JSON-RPC decodificado y devuelve la respuesta correspondiente, sin saber si llegó por stdio o por HTTP. Esto permite que el mismo servidor se exponga de forma remota en la segunda entrega sin reescribir la lógica de negocio.

7.1 Herramientas (tools) expuestas

Tool	Descripción
assess_symptoms	Evalúa síntomas reportados; devuelve condición probable, severidad, señales de alarma y clases terapéuticas sugeridas.
search_medications	Busca en el catálogo por texto libre y/o clase terapéutica.
get_medication_details	Ficha técnica completa de un SKU: dosis, contraindicaciones, efectos secundarios, precio.
check_interactions	Verifica interacciones fármaco-fármaco y conflictos de alergia contra medicamentos actuales.
check_stock	Consulta precio y existencias por sucursal o en todas las sucursales.
create_order	Genera un pedido; rechaza medicamentos con receta sin un <code>prescription_id</code> .
get_order_status	Consulta el estado de un pedido por su identificador.

7.2 Reglas de seguridad

- Solo se recomiendan medicamentos de venta libre, nunca se despacha un medicamento con receta sin un `prescription_id`.
- Cualquier señal de alarma (dolor de pecho, dificultad para respirar, sangre, desmayo, confusión, síntomas en menores de 2 años, más de 10 días de evolución) fuerza `severity`:

“urgent” y una recomendación explícita de buscar atención médica inmediata, en lugar de un medicamento.

- Toda evaluación de síntomas incluye un disclaimer explícito indicando que no es un diagnóstico médico.
- Estas reglas están implementadas en el dominio del servidor (`core/domain/triage.py` y `core/domain/orders.py`), no únicamente en el prompt del sistema, por lo que se cumplen independientemente de lo que el modelo decida hacer.

7.3 Modelo de datos

Los datos del catálogo viven en `src/servers/pharmacy/data/` y se cargan una vez por proceso, 10 medicamentos (8 de venta libre, 2 con receta), 6 mapeos de síntomas a condiciones más una lista de palabras clave de alarma, reglas de interacción fármaco-fármaco etiquetadas como “warning” o “blocking”, y 3 sucursales con existencias por SKU. Los pedidos se mantienen en memoria durante la vida del proceso y descuentan existencias de la sucursal correspondiente al confirmarse.

7.4 Ejemplo de intercambio JSON-RPC

Sollicitud (tools/call):

```
{"jsonrpc":"2.0","id":4,"method":"tools/call","params":
  {"name":"check_interactions","arguments":
    {"candidate_skus":["IBU400"],"current_medications":["warfarin"]}}
```

Respuesta:

```
{ "jsonrpc": "2.0", "id": 4, "result": { "content": [ { "type": "text", "text":  
    "\n\n            \\"safe_to_recommend\":" false,\n            "\blocking\":" [...]\n        } ],  
      "isError": false} }
```

isError permanece en false: la verificación se ejecutó correctamente, solo que el resultado indica que la combinación no es segura. isError: true se reserva para fallos de ejecución de

la herramienta (SKU desconocido, falta de receta, existencias insuficientes), siguiendo la convención de MCP de que esos casos son resultados normales que el modelo debe ver y ante los que debe reaccionar, no errores de transporte JSON-RPC.

8. Pruebas y validación

8.1 Suite automatizada

La suite de pruebas (tests/, ejecutada con pytest) contiene 32 casos, todos en estado passing:

- `test_jsonrpc.py`
 - Codificación/decodificación de los cuatro tipos de mensaje JSON-RPC, manejo de errores estándar (-32700, -32600, -32601), generación de ids y framing por línea.
- `test_pharmacy_tools.py`
 - Cada regla de dominio: coincidencia de condiciones por síntomas, señales de alarma (dolor de pecho, menores de 2 años, duración prolongada), bloqueo de interacciones (ibuprofeno + warfarina), conflicto de alergias, rechazo de pedidos con receta faltante o existencias insuficientes, y el recorrido completo del dispatcher (initialize → tools/list → tools/call).
- `test_client_stdio.py`
 - Prueba de integración real: levanta el servidor Pharmacy como subproceso genuino y lo recorre por el ciclo de vida MCP completo sobre stdin/stdout reales, sin ningún mock.

8.2 Pruebas manuales

Además de la suite automatizada, se ejecutaron pruebas manuales contra la API real de Anthropic (con clave de API provista por el estudiante) para validar el comportamiento observable, no solo unidades aisladas:

- Conexión simultánea a los tres servidores configurados (filesystem, git, pharmacy) sin errores.

- Triage completo de un caso realista (“rinorrea y dolor de garganta, 24 años, alergia a paracetamol”): el modelo encadenó `assess_symptoms` → `search_medications` → `check_interactions` → `check_stock` y evitó el alérgeno correctamente.
- Caso de señal de alarma (“dolor de pecho y dificultad para respirar”): el servidor forzó `severity urgent` y el modelo se negó a recomendar un medicamento, remitiendo a atención médica inmediata.
- Escenario combinado Filesystem + Git descrito en la sección 3.4, verificado contra el repositorio real con `git log`.
- El REPL real (`python -m src.main`) ejecutado de punta a punta de forma no interactiva, cubriendo `/servers`, `/tools`, `/log` y una conversación completa.
- Verificación bilingüe: el mismo flujo de preguntas y de triage funciona correctamente tanto en español como en inglés, respondiendo siempre en el idioma en que escribe el usuario.

9. Dificultades encontradas y cómo se resolvieron

- Falla al lanzar subprocessos con nombres de comando cortos en Windows (`FileNotFoundError` al invocar `npx`): `CreateProcess` de Windows no resuelve shims tipo `.cmd` por nombre corto de la misma forma en que lo hace una shell. Se resolvió usando `shutil.which()` para obtener la ruta completa del ejecutable antes de invocar `asyncio.create_subprocess_exec` en `src/mcp/transport/stdio.py`.
- `UnicodeEncodeError` en la consola de Windows: la codificación por defecto de la terminal (`cp1252`) no puede representar ciertos caracteres que el LLM puede incluir libremente en sus respuestas (marcas de verificación, símbolos, guiones largos), lo que

interrumpía el REPL a mitad de una conversación. Se resolvió reconfigurando `stdout/stderr` a UTF-8 con `errors="replace"` al iniciar `src/main.py`.

- La versión publicada del servidor oficial `mcp-server-git` no incluye una herramienta `git_init` (se confirmó inspeccionando el enum `GitTools` de su código fuente instalado), por lo que “crear un repositorio desde cero” no es posible usando únicamente los dos servidores oficiales. Se documentó como limitación conocida y se ajustó el guion de demostración para inicializar el repositorio de antemano.
- Namespacing de herramientas entre servidores: para permitir que varios servidores expongan tools con el mismo nombre corto sin colisionar, el registro (`src/host/registry.py`) antepone el nombre del servidor (`filesystem__read_file`, `pharmacy__check_stock`) antes de enviarlas al modelo, y revierte el prefijo al enrutar cada llamada de vuelta al cliente MCP correcto.

10. Lecciones aprendidas

- Implementar JSON-RPC y el ciclo de vida de MCP a mano (en lugar de usar un SDK) hizo mucho más claro por qué existen ciertas decisiones del protocolo, por ejemplo, por qué `notifications/initialized` no lleva `id` (es una notificación, no espera respuesta), o por qué `isError` en un resultado de tool es distinto de un error JSON-RPC a nivel de transporte.
- Probar contra servidores MCP oficiales reales (no solo el servidor propio) expuso problemas que un entorno completamente controlado no habría revelado, como las particularidades de lanzar subprocesos en Windows o las limitaciones reales del servidor de Git.

- Poner las reglas de seguridad del caso de uso de farmacia en el servidor (y no solo en el prompt del sistema) ayudó mucho, porque el comportamiento en casos de emergencia (negarse a recomendar ante una señal de alarma, exigir receta) no depende de que el modelo decida respetarlo.
- Mantener una separación estricta de capas (protocolo → MCP → LLM → host) facilitó agregar funcionalidad nueva sin tocar código no relacionado; por ejemplo, el cambio para responder en el idioma del usuario fue una sola línea en el system prompt del host, sin tocar el protocolo ni los servidores.

11. Referencias

Anthropic. (s. f.). *Messages API reference*. <https://docs.anthropic.com/>

GitHub. (s. f.). *Model Context Protocol servers* [Repositorio de código]. <https://github.com/modelcontextprotocol/servers>

Model Context Protocol. (2025, 25 de noviembre). *Model Context Protocol specification (2025-11-25)*. <https://modelcontextprotocol.io/specification/2025-11-25>

Model Context Protocol. (s. f.). *Architecture*. <https://modelcontextprotocol.io/docs/learn/architecture>

JSON-RPC Working Group. (s. f.). *JSON-RPC 2.0 specification*. <https://www.jsonrpc.org/specification>